



Spec-Driven Development with GitHub Spec Kit

From Intent to Implementation

Eric Greene · Xebia

Agenda

- Why a spec beats another prompt
- Anatomy of a Spec Kit spec
- Writing specs an agent can follow
- From raw intent to a reviewable spec
- Driving GitHub Copilot and Claude Code from the spec
- Treating specs as living documents

Why a spec beats another prompt

Prompt by prompt

Intent lives in a chat nobody
rereads

Each turn re-guesses what you
meant

Spec first

Intent lives in a file the agent
rereads

Each turn starts from the same
written goal

What that buys you

Drift shows up in review, not
production

Someone else can pick the feature
up

The Spec Kit workflow

- `/speckit-constitution` — rules that outlive one feature
- `/speckit-specify` and `/speckit-clarify` — what, why, and open questions
- `/speckit-plan` — how it's built, checked against the rules
- `/speckit-checklist` — unit tests for the English
- `/speckit-tasks` and `/speckit-analyze` — ordered work, then consistency
- `/speckit-implement` and `/speckit-converge` — build it, then close gaps

Demo 1 — From prompt to constitution

Demos 01 and 02 · about 18 minutes

Anatomy of a Spec Kit spec

- ## User Scenarios & Testing **(mandatory)** — P1, P2, P3
- Each story has Given / When / Then acceptance scenarios
- ## Requirements **(mandatory)** — numbered FR-001 onward
- ## Success Criteria **(mandatory)** — measurable, SC-001 onward
- ## Assumptions — defaults chosen instead of asking
- [NEEDS CLARIFICATION: ...] — the marker that stops guessing

Requirements an agent can act on

Too loose

Handle currency sensibly

Receipts are needed for large expenses

Approvals go to the right manager

Testable

FR-004: convert at the transaction date's rate

FR-006: require a receipt above 75 USD

FR-009: exit 0 clean, 1 violations, 2 bad input

From raw intent to a reviewable spec

- Start from one honest paragraph of intent
- `/speckit-specify` drafts the spec into `specs/001-.../`
- Unclear points are marked, not guessed
- `/speckit-clarify` asks up to five questions, one at a time
- Answers are written back into the requirements
- No git branch: `.specify/feature.json` tracks the feature

Demo 2 — From intent to a spec

Demo 03 · about 16 minutes

The plan and its guardrails

- `plan.md` answers how, after the spec answers what
- `## Technical Context` pins language, dependencies, constraints
- `## Constitution Check` rereads the rules you wrote first
- Gate: pass before research, recheck after design
- `## Complexity Tracking` records any rule you bend
- `/speckit-checklist` then tests the spec's English

Demo 3 — Plan, tasks, and analyze

Demo 04 · about 14 minutes

Driving the agent from the spec

- `/speckit-tasks` turns the plan into ordered, numbered work
- `/speckit-implement` works the list, not a chat history
- Scope a run to one story's tasks
- In Copilot, commands install as skills in `.github/skills/`
- Claude Code and others get the same commands
- Scope creep shows up as an unplanned task

Specs as living documents

- Requirement changed? Edit the spec first, then replan
- `/speckit-analyze` catches spec and tasks drifting apart
- `/speckit-converge` turns the gaps into new tasks
- Policy in data means many changes never touch code
- Put the spec diff in the same pull request
- The spec becomes onboarding for whoever comes next

Demo 4 — Implement and converge

Demos 05 and 06 · about 18 minutes

Questions and next steps

- Slides and all six demos are in the webinar repo
- Pin Spec Kit, then `specify init . --integration copilot`
- Write the constitution before your first spec
- Try it on one real feature, not a toy
- Put the spec in the pull request
- `eric.greene@xebia.com`